

Laboratory 2

Low-level Exploitations

In this lab, you will learn how to perform a **buffer overflow** attack. This is a phenomenon that occurs when a computer program writes data outside of a buffer allocated in memory. A buffer overflow can cause a crash or program hang leading to a denial of service (denial of service). Certain types of overflows, such as stack frame overflows, allow an attacker to load and execute arbitrary machine code on behalf of the program and with the rights of the account from which it is executed and thus gain a root shell.

Task 1 - Theory

1. What binary exploitation mitigation techniques do you know?
2. Did NX solve all binary attacks? Why?
3. Why do stack canaries end with 00?
4. What is NOP sled?

Task 2 - Binary attack warming up

We are going to work on a buffer overflow attack as one of the most popular and widely spread binary attacks. You are given a binary **warm_up**. [Link](#)

Answer the question and provide explanation details with PoC: "Why in the **warm_up** binary, opposite to the **binary64** from Lab1, the value of `i` doesn't change even if our input was very long?"

Task 3 - Linux local buffer overflow attack x86

You are given a very simple C code:

```
#include <stdio.h>
#include <string.h>

int main(int argc, char
*argv[]){ char buf[128];
strcpy(buf, argv[1]);
printf("%s\n", buf);
return 0;
}
```



1. Create a new file and put the code above into it:

```
touch source.c
```

2. Compile the file with C code in the binary with the following parameters in the case if you use x64 system:

```
gcc -o binary -fno-stack-protector -m32 -z execstack source.c
```

Questions:

- What does mean **-fno-stack-protector** parameter?
- What does mean **-m32** parameter?
- What does mean **-z execstack** parameter?

If you use x64 system, install the following package before compiling the program:

```
sudo apt install gcc-multilib
```

3. Disable ASLR before to start the buffer overflow attack:

```
sudo echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

4. Anyway, you can just download the [pre-compiled binary](#) task 3.

5. Choose any debugger to disassemble the binary - e.g. GNU debugger (**gdb**).

6. Perform the disassembly of the *required* function of the program.

7. Find the name and address of the *target* function. Copy the address of the function that follows (*is located below*) this function to jump across EIP.

8. Set the breakpoint with the assigned address.

9. Run the program with output that corresponds to the size of the buffer.

10. Examine the stack location and detect the start memory address of the buffer. In other words, this is the point where we break the program and rewrite the EIP.

11. Find the size of the writable memory on the stack. Re-run the same command as in step **#9** but without breakpoints. Increase the size of output symbols with several bytes until you get an overflow. In this way, iterate through different inputs to determine where the program crashes (segmentation fault). In simple words, perform a kind of fuzzing.



Task 4 - Catch the password x64

1. You are given a binary [Task5](#) compiled for 64-bit architecture. Try to exploit the program and find the correct password.

Questions:

2. What differences do you find between debugging 32-bit and 64-bit applications?
3. Do we need to use shellcode to complete this task? Justify your answer